

Software Requirements Specification

SmartStock

SME Retail Demand Forecasting & Inventory Optimization Platform

Prepared for: Software Engineering Project-Based Learning (5th Semester)

Guru Gobind Singh Indraprastha University (GGSIPU)

Document Version: 1.2

Date: 13/08/2026

IEEE 830-1998 Compliant SRS Format

Team Member	Role	Roll Number
Student 1 — Pratyush Jha	Data Engineering / ML / Analytics Lead	13718002724
Student 2 — Aayush Kumar Jha	Backend / API / System Design Lead	14418002724

Revision History

Version	Description	Date
1.0	Initial SRS draft covering requirements, use cases, and design constraints	12/08/2026
1.1	Requirements review: closed inventory/stock, PO lifecycle, and forecast-evaluation gaps (new FR-13/14/15, NFR-10/11, UC-4, current-stock formula, inventory_stock table)	12/08/2026
1.2	Reworded NFR-11 for audit clarity; added four future-scope bullets (manual stock adjustment, partial PO receipts, detailed supplier statuses, automated model retraining) — FROZEN for design phase	13/08/2026

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions (Summary)
 - 2.3 User Characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and Dependencies
- 3. Specific Requirements
 - 3.1 Functional Requirements
 - 3.2 Non-Functional Requirements
 - 3.3 Use Cases
 - 3.4 External Interface Requirements
 - 3.5 Testing-Relevant Requirement Notes
 - 3.5.1 Current Stock Calculation Rule
- 4. Data Requirements
 - 4.1 Conceptual Data Model (Star Schema)
 - 4.2 Data Retention
- 5. Future Scope and Open Issues
- Appendix A: Traceability Note

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for SmartStock, a demand-forecasting and inventory-optimization platform built for small and medium retail businesses. It is intended for use by the two-member development team, the course instructor/evaluator, and any future contributor who needs to understand the system's scope and behavior. This document follows the general structure recommended by IEEE Std 830-1998 for Software Requirements Specifications, adapted for an academic Project-Based Learning (PBL) deliverable.

1.2 Scope

SmartStock is a web-based application that allows a small retail business (e.g., a kirana store or a small retail chain) to: (a) record and manage product inventory, (b) ingest daily sales transaction data, (c) automatically forecast near-term demand for each product using a machine learning model, and (d) generate reorder alerts before stock runs out, based on forecasted demand, current stock level, and supplier lead time. The system also provides an analytics dashboard summarizing sales trends, stock turnover, and forecast accuracy for the store owner/manager.

The system is NOT intended to be a full point-of-sale (POS) or accounting system. It does not process live payments, integrate with tax-filing systems, or manage supplier contracts beyond basic lead-time metadata. These are explicitly out of scope for the current version, though they are noted as candidates for future work in Section 2.4.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SKU	Stock Keeping Unit — a unique identifier for a distinct product/variant.
ETL	Extract, Transform, Load — the process of ingesting raw transaction data, cleaning it, and loading it into the data warehouse.
Reorder Point	The stock level at which a new purchase order should be triggered to avoid a stockout, given lead time and forecasted demand.
Safety Stock	Extra inventory held to buffer against demand variability or supply delays.
MAPE	Mean Absolute Percentage Error — a metric used to evaluate forecasting model accuracy.
Lead Time	The time between placing a purchase order with a supplier and receiving the stock.
Fact Table / Dimension Table	Data-warehouse terms: a fact table stores measurable events (e.g., sales); a dimension table stores descriptive attributes (e.g., product, store, date).
MVP	Minimum Viable Product — the smallest feature set that delivers a usable, demonstrable system.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- K. K. Aggarwal and Yogesh Singh, "Software Engineering," New Age International, 3rd Ed., 2005 (course textbook).
- R. S. Pressman, "Software Engineering – A Practitioner's Approach," McGraw Hill, 5th Ed., 2001 (course textbook).
- FastAPI official documentation — <https://fastapi.tiangolo.com/>
- PostgreSQL official documentation — <https://www.postgresql.org/docs/>

1.5 Overview

The remainder of this document is organized as follows: Section 2 gives an overall description of the product, including stakeholders, product functions, constraints, and assumptions. Section 3 specifies detailed functional and non-functional requirements, including use cases, external interface requirements, and design constraints. Section 4 covers data requirements and the logical data model. Section 5 lists open issues and future scope. Appendices provide supporting detail.

2. Overall Description

2.1 Product Perspective

SmartStock is a new, self-contained system — it does not replace or integrate with any existing legacy system within the target business. It is a three-tier web application: a React-based frontend, a FastAPI backend exposing a REST API, and a PostgreSQL database. A separate scheduled batch process performs ETL and demand forecasting, writing results back to the database for the API layer to serve. The system is designed to be deployed as containerized services (Docker) on a cloud platform (e.g., Render/Railway) for demonstration and evaluation purposes.

2.2 Product Functions (Summary)

- User authentication and role-based access (Store Owner/Manager, Staff).
- Product and inventory master-data management (create/update/deactivate SKUs), including capturing and maintaining opening and current stock levels.
- Sales transaction ingestion (CSV upload or simulated feed) with validation and immediate stock decrement.
- ETL pipeline that cleans, transforms, and loads transaction data into a warehouse schema for forecasting and analytics (separate from live stock tracking).
- Per-SKU demand forecasting using historical sales data, with a low-confidence fallback for new/low-history SKUs.
- Reorder-point calculation engine using forecasted demand, current stock, and supplier lead time.
- Automated low-stock / reorder alerts.
- Analytics dashboard: sales trends, inventory turnover, stockout frequency, forecast accuracy.
- Purchase-order lifecycle: creation, receipt (with automatic stock replenishment), and cancellation.
- Forecast accuracy evaluation: comparing stored forecasts against actual sales once outcomes are known.

2.3 User Characteristics

User Class	Description	Technical Expertise
Store Owner / Manager	Owns or manages the retail business; primary consumer of alerts and analytics dashboard; makes reorder decisions.	Low to Medium — comfortable with basic web apps, spreadsheets.
Staff	Records/uploads daily sales data; views current stock and alerts.	Low — minimal technical background assumed.
System Administrator (dev team)	Manages users, monitors ETL jobs, maintains the system.	High — the development team itself during evaluation/demo.

2.4 Constraints

- Must be developed and deployed by a two-person team within approximately 4 months alongside regular coursework.
- Must use technologies the team can realistically master in this timeframe (React, FastAPI, PostgreSQL, scikit-learn/XGBoost) — no distributed data-processing frameworks (e.g., Spark, Kafka, Airflow), as they are unjustified at this data scale.
- Must be deployable on free/low-cost cloud tiers suitable for an academic demonstration.
- Sales data will be simulated/synthetic (or drawn from a public dataset), since real point-of-sale data from a live business is not available to the team.
- The system must remain explainable — forecasting and reorder logic must be interpretable enough to defend in a viva/demo, ruling out black-box deep learning approaches for the MVP.

2.5 Assumptions and Dependencies

- It is assumed that daily sales data will be available in a consistent, structured format (CSV) for ingestion.
- It is assumed that supplier lead time is a known, manually configured value per product (not dynamically sourced from a live supplier API).
- The system depends on a PostgreSQL instance being available and on scheduled jobs (via GitHub Actions or an equivalent scheduler) running reliably for nightly ETL/forecast refresh.
- It is assumed that forecast accuracy is acceptable if within a reasonable MAPE threshold for retail demand forecasting (typically 15–30% depending on SKU volatility); this will be validated empirically during development.

3. Specific Requirements

3.1 Functional Requirements

The functional requirements below are traceable to the use cases in Section 3.3 and will be used as the basis for test-case design (Section 3.5).

ID	Requirement Description	Priority	MVP
FR-1	The system shall allow a registered user to log in using email and password, and shall restrict access to features based on role (Owner/Manager vs. Staff).	High	Yes
FR-2	The system shall allow an Owner/Manager to create, update, and deactivate product (SKU) records, including name, category, unit cost, unit price, supplier lead time, and opening stock quantity at time of creation.	High	Yes
FR-3	The system shall allow Staff to upload daily sales transactions via a CSV file.	High	Yes
FR-4	The system shall validate uploaded transaction data and reject/flag rows with missing SKU references, negative quantities, or malformed dates.	High	Yes
FR-5	Upon successful validation of an uploaded transaction (FR-4), the system shall immediately decrement the affected SKU's current stock by the sold quantity, subject to NFR-10 and NFR-11 (the transaction itself is always recorded, even if it would drive stock negative). Separately, a scheduled ETL process shall clean, deduplicate, and load validated transactions into the warehouse fact table for forecasting and analytics; this batch process does not control stock levels.	High	Yes
FR-6	The system shall generate a demand forecast for each active SKU for a configurable forecast horizon (default: next 7 days), using historical sales data.	High	Yes
FR-7	The system shall compute a recommended reorder point for each SKU using forecasted demand, current stock level, supplier lead time, and a configurable safety-stock margin.	High	Yes
FR-8	The system shall generate an alert when a SKU's current stock falls at or below its computed reorder point.	High	Yes
FR-9	The system shall display a dashboard showing sales trends, current stock levels, active alerts, and inventory turnover for the logged-in user's store.	High	Yes
FR-10	The system shall allow an Owner/Manager to create a purchase order record directly from a reorder alert, specifying quantity ordered.	High	Yes
FR-11	The system shall display forecast-accuracy metrics (e.g., MAPE) for evaluation/demo purposes.	Medium	Yes

FR-12	The system shall support multiple stores under a single account.	Low	No — Future Scope
FR-13	The system shall allow an Owner/Manager to mark an open purchase order as Received, optionally editing the actual received quantity if it differs from the ordered quantity; upon receipt, the system shall increment the corresponding SKU's current stock by the received quantity. The system shall also allow an open purchase order to be marked Cancelled, which shall not affect stock.	High	Yes
FR-14	Once a forecasted date has passed and actual sales data for that date is available, the system shall compute forecast accuracy (MAPE) by comparing the stored forecast against actual demand, and shall make this value available to FR-11.	Medium	Yes
FR-15	If a SKU has fewer than a configurable minimum number of days of historical sales data (default: 14 days), the system shall generate its forecast using a simple moving-average fallback instead of the primary model, and shall flag that forecast as "low confidence."	High	Yes

3.2 Non-Functional Requirements

ID	Requirement Description	Category
NFR-1	The system shall respond to standard API requests (non-forecast) within 1 second under normal load for a single-store demo dataset.	Performance
NFR-2	Forecast/ETL batch jobs shall complete within a reasonable window (target: under 10 minutes for the demo dataset) and shall not block live API availability.	Performance
NFR-3	Passwords shall be stored using a salted hash (e.g., bcrypt); the system shall never store plaintext passwords.	Security
NFR-4	All API endpoints except login/registration shall require a valid authentication token.	Security
NFR-5	The system shall be usable by a non-technical store owner without training, using clear labels and a simple dashboard layout.	Usability
NFR-6	The system's codebase shall be modular (clear separation between API, business logic, ETL, and ML components) to support maintainability and future extension.	Maintainability
NFR-7	The system shall log ETL job failures and forecast-generation errors for later debugging.	Reliability
NFR-8	The system's database schema shall support adding new stores/products without requiring schema changes (extensibility).	Scalability

NFR-9	The system shall be deployable via a documented, repeatable process (Docker + CI configuration).	Portability / Deployability
NFR-10	The system shall treat each uploaded transaction row as idempotent with respect to stock decrement — re-uploading the same file or an overlapping file shall not decrement stock twice for the same transaction.	Data Integrity
NFR-11	The system shall never display a negative current-stock value. If a validated transaction would cause stock to become negative, the system shall set the displayed current stock to zero and flag the SKU for manual review, while preserving the transaction for audit purposes.	Reliability / Data Integrity

3.3 Use Cases

UC-1: Upload Sales Data

Field	Description
Actor	Staff / Owner-Manager
Precondition	User is authenticated and has Staff or Owner role.
Main Flow	1. User selects "Upload Sales" and chooses a CSV file. 2. System validates file format and row-level data. 3. System stores valid rows for ETL processing and reports any rejected rows with reasons.
Alternate Flow	If the file format is invalid, the system rejects the entire file with a clear error message before row-level validation.
Postcondition	Valid transactions are queued for the next ETL run.

UC-2: View Reorder Alerts

Field	Description
Actor	Owner/Manager
Precondition	ETL and forecasting jobs have run at least once; user is authenticated.
Main Flow	1. User opens the dashboard. 2. System displays a list of SKUs whose current stock is at or below the computed reorder point, along with forecasted demand and recommended order quantity. 3. User may create a purchase order directly from an alert.
Alternate Flow	If no SKUs are below their reorder point, the system displays an empty state with a confirmation message.
Postcondition	User has visibility into which products need reordering, and may act on it.

UC-3: Generate Demand Forecast (System-Triggered)

Field	Description
Actor	System (scheduled job)
Precondition	Warehouse tables contain sufficient historical sales data for a SKU (minimum data threshold configurable).
Main Flow	1. Scheduled job pulls cleaned sales history from the warehouse. 2. Job computes features (rolling averages, lag values, seasonality flags). 3. Job runs the forecasting model per SKU. 4. Job writes forecast results and updates reorder-point calculations.
Alternate Flow	If a SKU lacks sufficient history, the system falls back to a simple moving-average estimate and flags the forecast as "low confidence."
Postcondition	Updated forecasts and reorder points are available to the API layer.

UC-4: Receive Purchase Order

Field	Description
Actor	Owner/Manager
Precondition	An open purchase order exists for the SKU (created per FR-10).
Main Flow	1. User opens the purchase-order list and selects an open PO. 2. User marks it as "Received," confirming or editing the received quantity. 3. System increments the SKU's current stock by the received quantity (per FR-13). 4. System updates the PO status to "Received" and records the receipt timestamp.
Alternate Flow	User marks the PO as "Cancelled" instead of received; stock is unaffected and PO status is set to "Cancelled."
Postcondition	Stock reflects the newly received inventory (if received), and the PO no longer appears in the open-orders list.

3.4 External Interface Requirements

3.4.1 User Interfaces

A responsive web interface (React) providing: a login screen, a dashboard (sales trend chart, alerts list, stock overview), a product-management screen, a sales-upload screen, and a purchase-order screen.

3.4.2 Software Interfaces

- REST API (FastAPI) — JSON over HTTPS, documented via OpenAPI/Swagger (auto-generated).
- PostgreSQL database — accessed by both the API layer and the ETL/forecasting batch job.
- CI/CD — GitHub Actions for automated testing and scheduled ETL/forecast job execution.

3.4.3 Communication Interfaces

All client-server communication shall occur over HTTPS. The system does not require real-time (WebSocket) communication for the MVP.

3.5 Testing-Relevant Requirement Notes

Several functional requirements above contain multi-condition business logic well suited to structured testing techniques required by the course syllabus:

- FR-7 (reorder-point calculation) is suitable for Boundary Value Analysis (stock level exactly at, just above, and just below the reorder point) and Decision Table Testing (combinations of stock level, forecasted demand, and lead time).
- FR-4 (transaction validation) is suitable for Equivalence Class Testing (valid rows, missing-SKU rows, negative-quantity rows, malformed-date rows).
- UC-3's fallback logic (insufficient history → moving average) is suitable for Cause-Effect Graphing.
- The reorder-engine module (FR-7, FR-8) is a good candidate for Path Testing given its multiple conditional branches.

3.5.1 Current Stock Calculation Rule

To make FR-5, FR-7, FR-8, FR-13, and NFR-11 jointly testable, current stock is defined precisely as:

current_stock (SKU, store) = opening_stock (FR-2) + Σ received_quantity for all POs with status = Received (FR-13) - Σ quantity_sold for all validated transactions (FR-5), floored at 0 (NFR-11)

This formula is the basis for:

- Boundary Value Analysis on FR-8: stock exactly at, one above, and one below the reorder point.
- Decision Table Testing on FR-13: combinations of (PO status = Open/Received/Cancelled) × (received quantity = ordered / more / less than ordered).
- Equivalence Class Testing on FR-5/NFR-11: sale quantity less than, equal to, and greater than current stock (the last case exercises the floor-at-zero-and-flag rule, while confirming the original transaction is still recorded for audit).

4. Data Requirements

4.1 Conceptual Data Model (Star Schema)

The data warehouse layer uses a star schema centered on a sales fact table, supporting efficient aggregation for both the dashboard and the forecasting feature pipeline.

Fact Table: *fact_sales*

Column	Type	Description
sale_id	BIGINT (PK)	Unique transaction line identifier.
product_key	INT (FK)	References dim_product.
store_key	INT (FK)	References dim_store.
date_key	INT (FK)	References dim_date.
quantity_sold	INT	Units sold in this transaction line.
unit_price	NUMERIC	Price per unit at time of sale.
total_amount	NUMERIC	quantity_sold × unit_price.

Dimension Tables

- dim_product: product_key, sku, product_name, category, unit_cost, supplier_lead_time_days, reorder_safety_margin, is_active.
- dim_store: store_key, store_name, location, is_active.
- dim_date: date_key, full_date, day_of_week, week_of_year, month, quarter, is_weekend, is_holiday.

New Operational Table: *inventory_stock*

Current stock is operational, fast-changing data — it does not belong in a dimension table (dim_product is a slowly-changing dimension describing product attributes, not live quantities). It is tracked separately:

Column	Type	Description
inventory_id	INT (PK)	Unique record identifier.
product_key	INT (FK)	References dim_product.
store_key	INT (FK)	References dim_store.
opening_stock	INT	Stock quantity captured at product creation (FR-2).
current_stock	INT	Live stock quantity, maintained per the formula in Section 3.5.1. Never negative (NFR-11).
last_updated	TIMESTAMP	Last time current_stock changed (sale or PO receipt).

Supporting Tables (Operational / ML Output)

- forecast_results: forecast_id, product_key, store_key, forecast_date, predicted_demand, model_used, confidence_flag, generated_at.
- reorder_alerts: alert_id, product_key, store_key, current_stock, reorder_point, recommended_order_qty, status, created_at.
- users: user_id, name, email, password_hash, role, store_key.

Revised Table: purchase_orders

Column	Type	Description
po_id	BIGINT (PK)	Unique purchase order identifier.
product_key	INT (FK)	References dim_product.
store_key	INT (FK)	References dim_store.
quantity_ordered	INT	Quantity requested at PO creation (FR-10).
received_quantity	INT (nullable)	Actual quantity received (FR-13); null until receipt.
status	ENUM	Open / Received / Cancelled (explicit enum).
created_by	INT (FK → users)	User who created the PO.
created_at	TIMESTAMP	PO creation time.
received_at	TIMESTAMP (nullable)	Time the PO was marked Received; null until receipt.

4.2 Data Retention

Raw uploaded transaction files will be retained for at least one academic semester for reproducibility and audit purposes. Warehouse and forecast data will be retained for the life of the project.

5. Future Scope and Open Issues

- Multi-store comparison analytics (data model already supports this; UI/aggregation logic is future scope).
- Supplier-facing portal for automatically transmitting purchase orders.
- SMS/email notification delivery for reorder alerts (currently in-app only).
- Model selection automation (auto-choosing between moving-average, Prophet, and XGBoost per SKU based on historical accuracy).
- Anomaly/shrinkage detection (flagging unusual sales drops that may indicate theft or data-entry error).
- Manual stock adjustment / stocktake correction (e.g., for shrinkage, damage, or count discrepancies found during a physical audit).
- Partial purchase-order receipts (receiving a single PO across multiple deliveries/shipments).

- Detailed supplier PO statuses (e.g., Sent, Confirmed, In Transit) beyond the MVP's Open / Received / Cancelled.
- Automated model retraining or model selection driven by the forecast-accuracy metric (FR-14), rather than accuracy being a display-only metric.
- Open issue: final confirmation of which synthetic/public dataset will be used to seed realistic historical sales data for model training and demo purposes.
- Open issue: exact safety-stock formula (e.g., standard-deviation-based vs. fixed-percentage) to be finalized during design phase based on forecast-error characteristics observed in Sprint 1–2.

Appendix A: Traceability Note

Each functional requirement (FR-1 through FR-15) will be mapped to corresponding design elements (use cases, ER entities, API endpoints) and test cases in the Test Plan document, to be maintained as a separate requirements traceability matrix as the project progresses.